PROVE-IT BUILD PLAN

A build system for hackathons and portfolio projects

Merged from the Hack-Mystic Plan and the AI Software & Hackathon Building Playbook V2. Adds a kill-test stage, falsifiable experiments, and explicit time budgets.

PROBLEM → THESIS → KILL-TEST → PROOF DESIGN → VERTICAL SLICE → BREAK → PROVE → EXPLAIN → MARKET

Problem first, evidence second, product third.
AI handles ambiguity. Code handles authority.

STAGE 0

One-page brief (before any code)

If you can't answer all nine clearly, you're not ready to build.

#	Field	Answer this
1	Problem	[User] struggles with [problem] because [root cause].
2	Thesis	If we do X, the system achieves Y because Z. Must be falsifiable.
3	Invariant	What must never happen?
4	Why AI / agent	What does it decide? What does code enforce? Where does a human approve?
5	Load-bearing tech	What breaks if you remove it?
6	Most dangerous assumption	The thing that kills the project if false.
7	Proof experiment	Baseline → Intervention → Result.
8	Demo moment	What does a stranger remember after 60 seconds?
9	Distribution hook	What result is worth sharing publicly?

STAGE 1

Kill-test

Spend the first ~10% of your time trying to kill the idea.

- Test the dangerous assumption with a throwaway script, not the product.
- Read the official docs, limits, auth rules, pricing and quotas.

- If the assumption fails, **pivot the concept. Don't fight the protocol.**

STAGE 2

Architecture and vertical slice

Build one complete path that demonstrates the thesis.

> **User → Input → Agent → Tools → Deterministic validation → Action → Evidence → User**

- Keep the first architecture small. Justify every service.
- Use a real tool, a real decision and a real action.
- Label everything: **REAL / TESTNET / SIMULATED / DRY RUN / READ-ONLY / MEASURED**. Never present a simulation as a production result.
- Deploy the slice early. Test from a clean browser, fresh account and fresh environment.

STAGE 3

Break it

Attack your own product before anyone else does.

Area	Attack with
Input	Bad, missing, oversized, malformed or adversarial input
Auth	Unauthenticated, unauthorized, privilege escalation
State	Duplicate requests, stale data, partial updates, invalid transitions
Network	Timeout, dropped connection, rate limit, dependency outage
Data	Wrong type, missing record, inconsistent or corrupted data
AI	Hallucination, wrong classification, prompt failure, model outage, unsafe output
Money / permissions	Over-limit action, replay, unauthorized execution, wrong destination
Deployment	Environment mismatch, missing secret, bad build, broken integration
UX	Slow response, empty state, confusing error, inaccessible or mobile-broken flow

- **Output:** a “Ways I tried to break it” test matrix.
- The system must know **when not to act**, with a human escalation path.

STAGE 4

Prove it

Every important claim needs evidence another engineer can verify.

- Run the experiment, including the **removal test** (remove the load-bearing piece and show what breaks).
- Pick metrics that test the thesis, not ones that merely look impressive.

- Build an audit trail: input → evidence → decision → tool call → validation → action → result, with IDs, timestamps and tx hashes.
- Run a **judge attack** plus independent technical, security, sponsor and product reviews. Fix confirmed findings.

Proof pack checklist

Include	Include
One clean end-to-end demo path	Architecture diagram or data-flow note
Core test results	Live URL
Failure-case results	Real evidence (transactions, logs, API calls)
Real vs simulated labels	Known limitations and hardest engineering decision

STAGE 5

Explain and market

Make the project understandable without you in the room.

README structure

Thesis • Problem • Why existing solutions fail • Why agents • Architecture • Load-bearing integration • Proof experiment • Real vs simulated • Authority boundaries • Results • Live demo • Evidence • Local setup • Limitations

Demo (2–3 min)

Beat	Time
Problem	10–20s
Why existing approaches fail	10s
Solution and technical mechanism	30–50s
Live proof	60–90s
Failure boundary	10–20s
Evidence and result	20s

The demo should answer **“Why should I believe you?”**, not “What did you click?”

Marketing funnel

Post (problem first, tech last) → landing page → 60-second eval path → live product → proof → GitHub. One call to action per post. Polish the UI only after the proof exists.

GATES

Gates: every one gets a deliberate decision

Gate	Check	Passed
1	Problem explained in 20 seconds	[ ]
2	Thesis in one sentence	[ ]
3	Dangerous assumption tested	[ ]
4	Core flow works end-to-end	[ ]
5	Failure modes tested	[ ]
6	Claims reproducible	[ ]
7	Stranger can use it unaided	[ ]
8	Demo lands in 2–3 minutes	[ ]
9	Public post creates curiosity	[ ]

TIME BUDGETS

Default budgets: reshape them to fit your project

Phase	7-day hackathon	12-week portfolio
Brief + kill-test	20% (brief 10, kill-test 10)	Weeks 1–2
Slice + deploy	35%	Weeks 3–6 (slice and integrations)
Break	15%	Weeks 7–8
Prove	15%	Weeks 9–10
Package + demo	15%	Weeks 11–12 (explain and market)

The deadline is a checkpoint, not the end. You decide what ships by then; any gate not yet passed becomes the post-deadline roadmap. Portfolio mode: build 2–3 deep projects, not twelve shallow ones.

WHAT THIS ADDS

Over the two source documents

- **Kill-test stage** so bad ideas die early.
- **Falsifiable experiments and a removal test**, which the Mystic plan lacked.
- **Marketing, README and gates tied to proof**, which the Playbook lacked.
- **Time budgets as adjustable defaults**, with the deadline treated as a checkpoint for a project that keeps living.